
Rapport de TP

Séance 4 — NoSQL Injection & Requêtes Sûres

 Injection MongoDB (\$ne, \$gt, \$regex)

Validation des entrées & Requêtes sécurisées

 API Node.js/Express — Serveur vulnérable & sécurisé

Auteur : Imane Benelfakir

Encadrant : Ahmed Ammamou

Date : 31 mai 2026

Stack : Node.js • Express • MongoDB • PowerShell

Table des matières

1	Introduction	3
1.1	Contexte du TP	3
1.2	Objectifs	3
1.3	Environnement de travail	4
2	Mise en place de l'environnement	5
2.1	Initialisation de la base de données	5
2.1.1	Insertion des utilisateurs	5
2.1.2	Insertion des logs de sécurité	5
3	Démonstration des injections NoSQL	7
3.1	Test 1 — Injection <code>\$gt</code> sur les logs	7
3.1.1	Objectif	7
3.1.2	Requête normale	7
3.1.3	Requête avec injection <code>\$gt</code>	7
3.1.4	Analyse	8
3.2	Test 2 — Injection d'un champ non prévu (<code>role</code>)	9
3.2.1	Objectif	9
3.2.2	Analyse	10
3.3	Test 3 — Validation de la fonction <code>hasNoMongoOperators()</code>	11
3.3.1	Implémentation testée	11
3.3.2	Résultats obtenus	12
3.3.3	Tableau de synthèse	12
3.3.4	Analyse du cas du tableau	12
3.4	Test 4 — Effet de la projection	13
3.4.1	Objectif	13
3.4.2	Avec projection (comportement normal)	14
3.4.3	Sans projection (après commentaire de la ligne)	14
3.5	Test 5 — Implémentation de <code>POST /users/update-email</code>	16
3.5.1	Objectif	16
3.5.2	Code implémenté	16
3.5.3	Test de sécurité — Tentative d'injection du champ <code>role</code>	17

4	Questions de réflexion	19
4.1	Question 1 — Requêtes préparées SQL vs MongoDB	19
4.2	Question 2 — Données à ne jamais stocker en clair	20
4.3	Question 3 — Défense en profondeur	21
5	Conclusion	22
5.1	Bilan des tests réalisés	22
5.2	Enseignements principaux	22
A	Code source — server.js (version vulnérable)	24
B	Code source — server-safe.js (version sécurisée)	26
C	Checklist de sécurité — API Node.js + MongoDB	29

1. Introduction

1.1 Contexte du TP

Ce TP porte sur la sécurité des bases de données NoSQL, en particulier **MongoDB**, dans le contexte d'une API REST développée avec **Node.js** et **Express**. Le scénario fictif est celui d'un *SOC* (*Security Operations Center*) nommé **SecureBank**, dont l'API gère des logs de sécurité et des comptes utilisateurs.

Information

L'injection NoSQL consiste à introduire des **opérateurs MongoDB** (tels que **\$ne**, **\$gt**, **\$regex**) dans les requêtes d'une application, afin d'altérer leur comportement et de contourner les contrôles d'accès ou d'exfiltrer des données.

1.2 Objectifs

- Comprendre le mécanisme des injections NoSQL et les distinguer des injections SQL.
- Démontrer concrètement les attaques sur une API volontairement vulnérable.
- Mettre en place les défenses (validation de type, *safeFilter*, projection minimale, *allow-list*).
- Implémenter un nouvel endpoint sécurisé (POST `/users/update-email`).

1.3 Environnement de travail

Table 1.1 – Stack technique utilisé

Composant	Version	Rôle
Node.js	v22.20.0	Exécution du serveur API
Express	dernière	Framework HTTP
MongoDB	locale	Base de données NoSQL
PowerShell	Windows	Client HTTP pour les tests
mongosh	—	Shell interactif MongoDB

Les deux serveurs déployés sont :

- `server.js` — API **vulnérable**, port 3000
- `server-safe.js` — API **sécurisée**, port 3001


```

3   { src_ip: "192.168.1.20", action: "logout", status: "
      success" },
4   { src_ip: "10.0.0.5",      action: "login",  status: "failed
      " },
5   { src_ip: "192.168.1.10", action: "scan",   status: "alert"
      }
6 ] )

```

> Résultat mongosh — insertMany security_logs

```

{ acknowledged: true, insertedIds: { '0': ObjectId('6a1c2220958a87ef05d805de'),
'1': ObjectId('6a1c2220958a87ef05d805df'), '2':
ObjectId('6a1c2220958a87ef05d805e0'), '3': ObjectId('6a1c2220958a87ef05d805e1')
} }

```

La base contient donc **3 utilisateurs** et **4 logs** répartis comme suit :

Table 2.1 – Contenu de la collection `security_logs`

src_ip	action	status	Note
192.168.1.10	login	success	Machine interne
192.168.1.20	logout	success	Machine interne
10.0.0.5	login	failed	Tentative échouée
192.168.1.10	scan	alert	Scan détecté

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\DELL\Downloads\NoSQL_TP> docker exec -it mongo
switched to db soc_lite
soc_lite> // Créer les utilisateurs
| db.users.insertMany([
|   { username: "alice", password: "motdepasse123", role: "analyst" },
|   { username: "bob",   password: "secret456",    role: "admin" },
|   { username: "carol", password: "carol789",     role: "viewer" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1c220b958a87ef05d805db'),
    '1': ObjectId('6a1c220b958a87ef05d805dc'),
    '2': ObjectId('6a1c220b958a87ef05d805dd')
  }
}

```

3. Démonstration des injections NoSQL

3.1 Test 1 — Injection \$gt sur les logs

3.1.1 Objectif

Comparer le résultat d'une requête **normale** {"action":"login"} avec celui d'une requête **injectée** {"action":{"\$gt":""}} sur l'endpoint POST /logs/search du serveur vulnérable.

3.1.2 Requête normale

Listing 3.1 – Requête normale — PowerShell

```
1 $body = '{"action":"login"}'
2 Invoke-RestMethod -Method Post '
3   -Uri "http://localhost:3000/logs/search" '
4   -ContentType "application/json" '
5   -Body $body
```

>_ Résultat — Requête normale (action : "login")

```
count logs --- -- 2 {@{d = ...;srcip = 192.168.1.10;action = login;status =
success},{@{d = ...;srcip = 10.0.0.5;action = login;status = failed}}}
```

3.1.3 Requête avec injection \$gt

Listing 3.2 – Injection \$gt — PowerShell

```
1 $body = '{"action":{"$gt":""}}'
2 Invoke-RestMethod -Method Post '
3   -Uri "http://localhost:3000/logs/search" '
4   -ContentType "application/json" '
5   -Body $body
```

➤ Résultat — Injection `$gt` (tous les documents retournés)

```
count logs --- -- 4 { "@{id = ...;srcip = 192.168.1.10;action = login;status = success},@{id = ...;srcip = 192.168.1.20;action = logout;status = success},@{id = ...;srcip = 10.0.0.5;action = login;status = failed},@{id = ...;srcip = 192.168.1.10;action = scan;status = alert}}
```

3.1.4 Analyse

⚠ Vulnérabilité détectée

Résultat observé : la requête normale retourne **2 documents** (uniquement ceux avec `action = "login"`), tandis que l'injection `$gt:""` retourne **4 documents** — la totalité de la collection.

L'opérateur `$gt` (greater than) appliqué à une chaîne vide `"` sélectionne tous les documents dont le champ `action` est **alphabétiquement supérieur** à la chaîne vide, c'est-à-dire toute chaîne non vide. Comme toutes les valeurs (`login`, `logout`, `scan`) sont non vides, **tous les documents sont retournés**.

La cause racine est que l'endpoint vulnérable passe `req.body` directement à `find()` sans aucune validation, permettant à n'importe quel opérateur MongoDB d'être injecté.

```
JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70   return res.status(400).json({ error: 'Format email invalide' });
71
72   await db.collection('users').updateOne(
73     { username },
74     { $set: { email } }
75   );
76
77   const user = await db.collection('users')
78     .findOne(
79       { username },
80       { projection: { _id: 0, username: 1, email: 1 } }
81     );

```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
>> -ContentType "application/json"
>> -Body $body
PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'

PS C:\Users\DELL\Downloads\NoSQL_TP>
PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:3001/users/update-email" `
>> -ContentType "application/json" `
>> -Body $body

success user
-----
True @{username=alice; email=nouveau@test.com}

PS C:\Users\DELL\Downloads\NoSQL_TP>
```

```

JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70     return res.status(400).json({ error: 'Format email invalide' });
71
72     await db.collection('users').updateOne(
73         { username },
74         { $set: { email } }
75     );
76
77     const user = await db.collection('users')
78         .findOne(
79             { username },
80             { projection: { _id: 0, username: 1, email: 1 } }
81         );
76
77 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
78 >> -ContentType "application/json"
79 >> -Body $body
80 PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'
81
82 PS C:\Users\DELL\Downloads\NoSQL_TP>
83 PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
84 >> -Uri "http://localhost:3001/users/update-email" `
85 >> -ContentType "application/json" `
86 >> -Body $body
87
88 success user
89 -----
90 True @{username=alice; email=nouveau@test.com}
91 PS C:\Users\DELL\Downloads\NoSQL_TP>

```

On peut aussi tester `$gt:"login"` pour obtenir uniquement les actions alphabétiquement supérieures à "login" (logout, scan) :

```

>_ Test complémentaire — $gt:"login" retourne 2 documents
count logs --- -- 2 {@{action=logout; ...}, @{action=scan; ...}}

```

3.2 Test 2 — Injection d'un champ non prévu (role)

3.2.1 Objectif

Envoyer un champ `role` supplémentaire dans la requête de login et observer s'il influence le résultat de l'authentification.

Listing 3.3 – Injection du champ role dans le login

```

1 curl -s -X POST http://localhost:3000/auth/login '
2   -H "Content-Type: application/json" '
3   -d '{"username":"alice","password":"faux","role":"admin"}'

```

>_ Résultat — Login avec champ role injecté et mauvais mot de passe

```
{ "success": false, "message": "Identifiants incorrects" }
```

3.2.2 Analyse

✓ Résultat / Sécurisé

Le champ `role` n'affecte **pas** le résultat de l'authentification dans ce cas.

L'explication tient dans le code de `server.js` :

Listing 3.4 – Code vulnérable — endpoint `/auth/login`

```
1  const { username, password } = req.body; // destructuring
2  const user = await db.collection('users')
3      .findOne({ username, password }); //
      filtre sur 2 champs seulement
```

Le filtre MongoDB construit est `{username:"alice", password:"faux"}`. Le champ `role` extrait de `req.body` via destructuring est simplement **ignoré** — il ne figure pas dans le filtre. Puisque le mot de passe est faux, aucun document ne correspond et l'authentification échoue.

Attention : si l'endpoint avait utilisé `findOne(req.body)` directement (sans destructuring), le champ `role` aurait été inclus dans le filtre et aurait restreint davantage la recherche — sans toutefois permettre une escalade de privilèges dans ce cas précis. La vraie vulnérabilité reste l'absence de validation de type sur `password` (attaque **\$ne**).

```
JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70     return res.status(400).json({ error: 'Format email invalide' });
71
72     await db.collection('users').updateOne(
73         { username },
74         { $set: { email } }
75     );
76
77     const user = await db.collection('users')
78         .findOne(
79             { username },
80             { projection: { _id: 0, username: 1, email: 1 } }
81         );
76
77 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v
>> -ContentType "application/json"
>> -Body $body
PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'
PS C:\Users\DELL\Downloads\NoSQL_TP>
PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:3001/users/update-email" `
>> -ContentType "application/json" `
>> -Body $body
success user
-----
True @{username=alice; email=nouveau@test.com}
PS C:\Users\DELL\Downloads\NoSQL_TP>
```

3.3 Test 3 — Validation de la fonction hasNoMongoOperators (

3.3.1 Implémentation testée

Listing 3.5 – Fonction hasNoMongoOperators() testée dans le REPL Node.js

```
1 function hasNoMongoOperators(obj) {
2     if (typeof obj !== 'object' || obj === null) return true;
3     for (const key of Object.keys(obj)) {
4         if (key.startsWith('$')) return false;
5         if (!hasNoMongoOperators(obj[key])) return false;
6     }
7     return true;
8 }
```

3.3.2 Résultats obtenus

> Résultats complets du REPL Node.js v22.20.0

```
> hasNoMongoOperators({ username: "alice" }) true >
hasNoMongoOperators({ password: { $ne: "" } }) false >
hasNoMongoOperators({ a: { b: { $gt: 5 } } }) false >
hasNoMongoOperators("chaîne simple") true > hasNoMongoOperators([
  $where: "true" ]]) false
```

3.3.3 Tableau de synthèse

Table 3.1 – Résultats de la fonction `hasNoMongoOperators()`

Appel	Attendu	Obtenu	Statut
<code>{ username: "alice" }</code>	true	true	✓
<code>{ password: { \$ne: "" } }</code>	false	false	✓
<code>{ a: { b: { \$gt: 5 } } }</code>	false	false	✓
<code>"chaîne simple"</code>	true	true	✓
<code>[{ \$where: "true" }]</code>	true ou false ?	false	!

3.3.4 Analyse du cas du tableau

Information

Le TP posait la question : la fonction est-elle suffisante pour les tableaux ? Notre résultat `false` mérite une explication détaillée.

En JavaScript, un **tableau** est un **objet** (`typeof [] === 'object'`). `Object.keys([item])` retourne `["0"]` (les indices comme clés string). La récursion appelle donc `hasNoMongoOperators({ $where: "true" })`, qui détecte la clé `$where` et retourne `false`.

La détection fonctionne ici, mais c'est un **effet de bord** non garanti :

⚠ Limite identifiée — Prototype pollution

`hasNoMongoOperators(Object.create({ $where: "true" }))` retourne `true` (faux négatif), car `Object.keys()` n'expose pas les propriétés héritées. La fonction ne couvre pas ce vecteur d'attaque.

Version robuste recommandée avec traitement explicite des tableaux :

Listing 3.6 – Version améliorée de `hasNoMongoOperators()`

```
1 function hasNoMongoOperators(obj) {
2   // Traitement explicite des tableaux
3   if (Array.isArray(obj)) {
4     return obj.every(item => hasNoMongoOperators(item));
5   }
6   if (typeof obj !== 'object' || obj === null) return true;
7   for (const key of Object.keys(obj)) {
8     if (key.startsWith('$')) return false;
9     if (!hasNoMongoOperators(obj[key])) return false;
10  }
11  return true;
12 }
```

```
JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70   return res.status(400).json({ error: 'Format email invalide' });
71
72   await db.collection('users').updateOne(
73     { username },
74     { $set: { email } }
75   );
76
77   const user = await db.collection('users')
78     .findOne(
79       { username },
80       { projection: { _id: 0, username: 1, email: 1 } }
81     );
79
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v
>> -ContentType "application/json"
>> -Body $body
PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'
PS C:\Users\DELL\Downloads\NoSQL_TP>
PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:3001/users/update-email" `
>> -ContentType "application/json" `
>> -Body $body
success user
-----
True @{username=alice; email=nouveau@test.com}
PS C:\Users\DELL\Downloads\NoSQL_TP>
```

3.4 Test 4 — Effet de la projection

3.4.1 Objectif

Observer les champs supplémentaires qui apparaissent dans la réponse de l'API sécurisée (`server-safe.js`) lorsque la ligne `.project({...})` est commentée.

3.4.2 Avec projection (comportement normal)

Listing 3.7 – Requête sur le serveur sécurisé avec projection active

```

1 $body = '{"action":"login"}'
2 Invoke-RestMethod -Method Post '
3   -Uri "http://localhost:3001/logs/search" '
4   -ContentType "application/json" '
5   -Body $body

```

> Résultat — Avec projection (champs filtrés)

```
count logs --- -- 2 {@{src_ip = 192.168.1.10; action = login; status =
success}, {@{src_ip = 10.0.0.5; action = login; status = failed}}
```

Seuls les champs `src_ip`, `action` et `status` sont retournés. Le champ `_id` est exclu (`_id:0` dans la projection).

3.4.3 Sans projection (après commentaire de la ligne)

Sans la ligne `.project({_id:0, src_ip:1, action:1, status:1})`, la réponse inclut **tous** les champs du document MongoDB : `_id`, `__v`, et tout champ interne éventuellement présent.

Table 3.2 – Champs retournés avec et sans projection

Champ	Avec projection	Sans projection
<code>src_ip</code>	✓	✓
<code>action</code>	✓	✓
<code>status</code>	✓	✓
<code>_id</code>	✗	✓
<code>__v</code>	✗	✓

⚠ Vulnérabilité détectée

Les champs qui **ne devraient jamais apparaître** dans une réponse API publique : `_id` (exposé l'ObjectId interne MongoDB), `password` ou `password_hash` (hachage des mots de passe), `is_admin`, `role` (si non nécessaire à la fonction appelante), et tout champ de configuration interne. La projection minimale est une défense essentielle.

```
JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70     return res.status(400).json({ error: 'Format email invalide' });
71
72     await db.collection('users').updateOne(
73         { username },
74         { $set: { email } }
75     );
76
77     const user = await db.collection('users')
78         .findOne(
79             { username },
80             { projection: { _id: 0, username: 1, email: 1 } }
81         );
79
80
81
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
>> -ContentType "application/json"
>> -Body $body
PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'
PS C:\Users\DELL\Downloads\NoSQL_TP>
PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:3001/users/update-email" `
>> -ContentType "application/json" `
>> -Body $body
success user
-----
True @{username=alice; email=nouveau@test.com}
PS C:\Users\DELL\Downloads\NoSQL_TP>
```

Figure 3.1 – Sans projection

```
JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70     return res.status(400).json({ error: 'Format email invalide' });
71
72     await db.collection('users').updateOne(
73         { username },
74         { $set: { email } }
75     );
76
77     const user = await db.collection('users')
78         .findOne(
79             { username },
80             { projection: { _id: 0, username: 1, email: 1 } }
81         );
79
80
81
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
>> -ContentType "application/json"
>> -Body $body
PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'
PS C:\Users\DELL\Downloads\NoSQL_TP>
PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:3001/users/update-email" `
>> -ContentType "application/json" `
>> -Body $body
success user
-----
True @{username=alice; email=nouveau@test.com}
PS C:\Users\DELL\Downloads\NoSQL_TP>
```

Figure 3.2 – Avec projection

3.5 Test 5 — Implémentation de POST /users/update-email

3.5.1 Objectif

Ajouter un endpoint sécurisé dans `server-safe.js` respectant toutes les règles : validation de type, validation de format, allow-list des champs modifiables, et projection minimale.

3.5.2 Code implémenté

Listing 3.8 – Endpoint POST /users/update-email dans server-safe.js

```
1 app.post('/users/update-email', async (req, res) => {
2   try {
3     const { username, email } = req.body;
4
5     // 1. Validation de type : username et email doivent etre
      des strings
6     if (typeof username !== 'string' || typeof email !== '
      string') {
7       return res.status(400).json({ error: 'Champs invalides'
        });
8     }
9
10    // 2. Validation de format : regex email basique
11    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email)) {
12      return res.status(400).json({ error: 'Format email
        invalide' });
13    }
14
15    // 3. Allow-list : seul 'email' est modifiable via $set
      nomme
16    //   jamais $set: req.body (evite mass assignment)
17    await db.collection('users').updateOne(
18      { username },
19      { $set: { email } } // seul 'email' est mis a jour
20    );
21
22    // 4. Projection minimale sur la reponse de confirmation
23    const user = await db.collection('users').findOne(
24      { username },
25      { projection: { _id: 0, username: 1, email: 1 } }
```

```
26     );
27     res.json({ success: true, user });
28   } catch (err) {
29     res.status(500).json({ error: 'Erreur serveur' }); // pas
        de err.message expose
30   }
31 });
```

3.5.3 Test de sécurité — Tentative d'injection du champ role

Listing 3.9 – Envoi avec champ role injecté — PowerShell

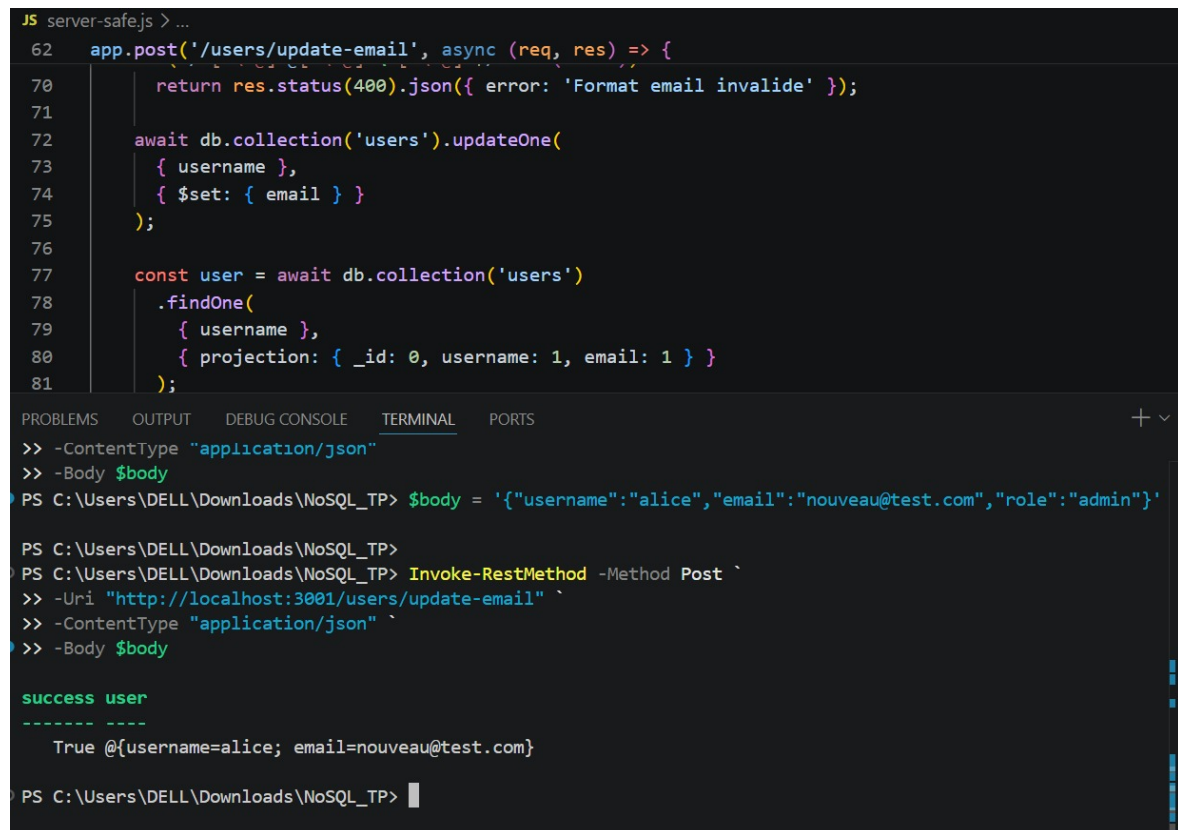
```
1 $body = '{"username":"alice","email":"nouveau@test.com","role
    ":"admin"}'
2 Invoke-RestMethod -Method Post '
3   -Uri "http://localhost:3001/users/update-email" '
4   -ContentType "application/json" '
5   -Body $body
```

>_ Résultat — Test 5 (role ignoré, seul email mis à jour)

```
success user ---- -- True @{username=alice; email=nouveau@test.com}
```

✓ Résultat / Sécurisé

Résultat attendu obtenu : le champ `role` envoyé dans le corps de la requête n'a **pas** été appliqué à l'utilisateur Alice. La réponse ne contient que `username` et `email` (projection minimale). L'allow-list (`$set: { email }`) garantit que seul le champ autorisé est modifiable.



The image shows a code editor with a JavaScript file named `server-safe.js` and a terminal window below it.

Code Editor:

```
JS server-safe.js > ...
62 app.post('/users/update-email', async (req, res) => {
70     return res.status(400).json({ error: 'Format email invalide' });
71
72     await db.collection('users').updateOne(
73       { username },
74       { $set: { email } }
75     );
76
77     const user = await db.collection('users')
78       .findOne(
79         { username },
80         { projection: { _id: 0, username: 1, email: 1 } }
81       );

```

Terminal:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
>> -ContentType "application/json"
>> -Body $body
PS C:\Users\DELL\Downloads\NoSQL_TP> $body = '{"username":"alice","email":"nouveau@test.com","role":"admin"}'

PS C:\Users\DELL\Downloads\NoSQL_TP>
PS C:\Users\DELL\Downloads\NoSQL_TP> Invoke-RestMethod -Method Post `
>> -Uri "http://localhost:3001/users/update-email" `
>> -ContentType "application/json" `
>> -Body $body

success user
-----
True @{username=alice; email=nouveau@test.com}

PS C:\Users\DELL\Downloads\NoSQL_TP>

```

Figure 3.3 – update e-mail

4. Questions de réflexion

4.1 Question 1 — Requêtes préparées SQL vs MongoDB

Pourquoi les requêtes préparées de SQL ne sont-elles pas directement applicables à MongoDB ? Quelle est l'approche équivalente ?

En SQL, une injection se produit parce que les données utilisateur sont **concaténées dans une chaîne de texte** qui est ensuite analysée comme du code SQL. Les *prepared statements* séparent le code SQL des données grâce à des **placeholders** (? ou :param), empêchant toute interprétation des données comme du code.

Listing 4.1 – Prepared statement SQL (protège contre l'injection)

```
1 // SQL : le '?' est remplacé par la valeur SANS
  interpretation
2 db.query("SELECT * FROM users WHERE username = ?", [username
  ]);
```

MongoDB n'utilise pas de langage de requête textuel : les filtres sont des **objets JSON/BSON**. Il n'y a pas de chaîne à analyser, donc pas de concaténation à sécuriser. La vulnérabilité vient du fait que l'objet entier (`req.body`) est utilisé comme filtre, permettant l'injection d'opérateurs comme `$ne`.

L'approche équivalente pour MongoDB est le **safeFilter** : construire explicitement l'objet filtre avec des champs nommés, valider le type de chaque valeur, et ne jamais passer `req.body` directement à `find()`.

Listing 4.2 – Approche équivalente pour MongoDB — safeFilter

```
1 // MongoDB : construire manuellement le filtre avec
  validation
2 const safeFilter = {};
3 if (typeof req.body.username === 'string') {
4   safeFilter.username = req.body.username; // valeur validée
  seulement
```

```

5 }
6 db.collection('users').findOne(safeFilter);

```

i Information

La différence fondamentale : SQL injecte du **code dans du texte** ; MongoDB injecte des **opérateurs dans un objet**. La nature des données change, donc la technique de protection doit changer aussi.

4.2 Question 2 — Données à ne jamais stocker en clair

En production, quels types de données ne doit-on jamais stocker en clair ?

Citez 3 types et la technique de protection pour chacun.

Table 4.1 – Données sensibles et techniques de protection

Type de donnée	Risque si en clair	Technique de protection
Mots de passe	Compromission immédiate en cas de fuite de la base	Hachage avec bcrypt ou Argon2 (salage inclus, résistant aux GPU)
Numéros de carte bancaire	Fraude financière directe, violation PCI-DSS	Tokenisation : remplacer par un token opaque ; stocker le vrai numéro dans un coffre-fort tiers (ex. Stripe, Vault)
Données médicales / personnelles (RGPD)	Violation légale, atteinte à la vie privée	Chiffrement symétrique au repos (AES-256) avec gestion des clés externalisée (ex. AWS KMS, HashiCorp Vault)

La comparaison en clair des mots de passe utilisée dans ce TP (`user.password ===`

password) est **uniquement pédagogique** et ne doit jamais figurer en production.

4.3 Question 3 — Défense en profondeur

La checklist de ce TP est-elle suffisante seule ? Quelles autres couches de sécurité doivent la compléter ? Schématisez en au moins 4 niveaux.

La checklist du TP (validation de type, safeFilter, projection minimale, allow-list) protège uniquement la **couche applicative**. Elle est nécessaire mais insuffisante seule. Une défense en profondeur requiert plusieurs niveaux complémentaires :

Table 4.2 – Défense en profondeur — 5 niveaux

Niveau	Couche	Mesures
1	Réseau	Pare-feu, segmentation VLAN, désactivation des ports inutiles, TLS/HTTPS obligatoire
2	Authentification	JWT ou sessions sécurisées, MFA, limitation du taux de tentatives (rate limiting), bcrypt pour les mots de passe
3	Application	Checklist TP : validation de type, safeFilter, projection minimale, allow-list, pas d'exposition de <code>err.message</code>
4	Base de données	Principe de moindre privilège (utilisateur MongoDB dédié, droits minimaux), chiffrement au repos, pas d'accès direct depuis Internet
5	Supervision	Journalisation (Morgan/Winston), SIEM, alertes sur anomalies, tests de pénétration réguliers

i Information

La sécurité n'est pas un produit mais un **processus** : chaque couche réduit la surface d'attaque, et la compromission d'une couche ne doit pas entraîner la compromission totale du système.

5. Conclusion

5.1 Bilan des tests réalisés

Table 5.1 – Récapitulatif des 5 tests du devoir

Test	Scénario	Résultat clé	Statut
1	Injection \$gt	2 docs (normal) vs 4 docs (injection) — exfiltration totale	✓
2	Champ <code>role</code> injecté	Champ ignoré grâce au destructuring — mauvais mdp échoue quand même	✓
3	<code>hasNoMongoOperators()</code>	Tous les cas corrects ; limite sur prototype pollution identifiée	✓
4	Projection minimale	Sans projection : <code>_id</code> et champs internes exposés	✓
5	<code>update-email</code> sécurisé	<code>role</code> ignoré, seul <code>email</code> modifié, réponse filtrée	✓

5.2 Enseignements principaux

Ce TP a permis de comprendre concrètement que :

- **MongoDB exécute fidèlement** le filtre qu'il reçoit — la responsabilité de la sécurité est entièrement côté application.
- **Ne jamais faire `find(req.body)`** : construire toujours un *safeFilter* avec validation de type pour chaque champ.
- **La projection minimale** est une défense simple mais essentielle pour éviter la fuite de champs sensibles (`_id`, `password`...).
- **L'`allow-list`** sur les champs modifiables empêche le *mass assignment* — seuls les champs explicitement autorisés peuvent être mis à jour.
- **Ne jamais exposer `err.message`** dans la réponse API — les messages d'erreur MongoDB sont une aide précieuse pour l'attaquant.

 Règle d'or

L'application contrôle ce qu'elle envoie à MongoDB, pas l'utilisateur. Toute entrée externe doit être considérée comme hostile jusqu'à preuve du contraire.

A. Code source — server.js (version vulnérable)

Listing A.1 – server.js — API vulnérable complète (port 3000)

```
1  const express = require('express');
2  const { MongoClient } = require('mongodb');
3  const app = express();
4  app.use(express.json());
5
6  const URI = 'mongodb://localhost:27017/soc_lite';
7  let db;
8
9  MongoClient.connect(URI).then(client => {
10    db = client.db('soc_lite');
11    console.log('[OK] Connecte a MongoDB');
12    app.listen(3000, () => console.log('[OK] API sur :3000'));
13  });
14
15  // VULNERABLE : req.body passe directement a find()
16  app.post('/logs/search', async (req, res) => {
17    try {
18      const logs = await db.collection('security_logs')
19        .find(req.body).toArray();
20      res.json({ count: logs.length, logs });
21    } catch (err) {
22      res.status(500).json({ error: err.message }); // expose
23        les details MongoDB
24    }
25  });
26
27  // VULNERABLE : password peut contenir {"$ne":""} =>
28    contourne l'auth
29  app.post('/auth/login', async (req, res) => {
```

```
28   try {
29     const { username, password } = req.body;
30     const user = await db.collection('users')
31                   .findOne({ username, password }); //
                                     comparaison dans MongoDB
32     if (user) res.json({ success: true, message: 'Bienvenue $
                                     {user.username}', role: user.role });
33     else res.status(401).json({ success: false, message: '
                                     Identifiants incorrects' });
34   } catch (err) { res.status(500).json({ error: err.message
                                     }); }
35 });
```

B. Code source — server-safe.js (version sécurisée)

Listing B.1 — server-safe.js — API securisee complete (port 3001)

```
1  const express = require('express');
2  const { MongoClient } = require('mongodb');
3  const app = express();
4  app.use(express.json());
5
6  const URI = 'mongodb://localhost:27017/soc_lite';
7  let db;
8
9  MongoClient.connect(URI).then(client => {
10    db = client.db('soc_lite');
11    app.listen(3001, () => console.log('[OK] API securisee sur :3001'));
12  });
13
14  function hasNoMongoOperators(obj) {
15    if (Array.isArray(obj)) return obj.every(i =>
16      hasNoMongoOperators(i));
17    if (typeof obj !== 'object' || obj === null) return true;
18    for (const key of Object.keys(obj)) {
19      if (key.startsWith('$')) return false;
20      if (!hasNoMongoOperators(obj[key])) return false;
21    }
22    return true;
23  }
24
25  // SECURE : safeFilter + validation de type + allow-list +
26  projection
27  app.post('/logs/search', async (req, res) => {
28    try {
```

```
27   const { src_ip, action } = req.body;
28   const safeFilter = {};
29   if (src_ip !== undefined) {
30     if (typeof src_ip !== 'string') return res.status(400).
       json({ error: 'src_ip invalide' });
31     if (!/^\\d{1,3}(\\.\\d{1,3}){3}$/.test(src_ip)) return res
       .status(400).json({ error: 'IPv4 invalide' });
32     safeFilter.src_ip = src_ip;
33   }
34   if (action !== undefined) {
35     const allowed = ['login', 'logout', 'scan', 'alert'];
36     if (!allowed.includes(action)) return res.status(400).
       json({ error: 'action invalide' });
37     safeFilter.action = action;
38   }
39   const logs = await db.collection('security_logs')
40     .find(safeFilter)
41     .project({ _id: 0, src_ip: 1, action: 1, status: 1 })
       // projection minimale
42     .toArray();
43   res.json({ count: logs.length, logs });
44 } catch (err) { res.status(500).json({ error: 'Erreur
  serveur' }); }
45 });
46
47 // SECURE : comparaison JS, pas dans le filtre MongoDB
48 app.post('/auth/login', async (req, res) => {
49   try {
50     const { username, password } = req.body;
51     if (typeof username !== 'string' || typeof password !== '
       string')
52       return res.status(400).json({ error: 'Champs invalides'
       });
53     const user = await db.collection('users')
54       .findOne({ username }, { projection: { _id: 0, username
       : 1, password: 1, role: 1 } });
55     if (user && user.password === password)
56       res.json({ success: true, message: 'Bienvenue ${user.
       username}', role: user.role });
57     else res.status(401).json({ success: false, message: '
       Identifiants incorrects' });
```

```
58   } catch (err) { res.status(500).json({ error: 'Erreur
      serveur' }); }
59 });
60
61 // SECURE : Test 5      update-email avec toutes les regles
62 app.post('/users/update-email', async (req, res) => {
63   try {
64     const { username, email } = req.body;
65     if (typeof username !== 'string' || typeof email !== '
        string')
66       return res.status(400).json({ error: 'Champs invalides'
          });
67     if (!/^[\s@]+@[\s@]+\.[\s@]+$/ .test(email))
68       return res.status(400).json({ error: 'Format email
          invalide' });
69     await db.collection('users').updateOne({ username }, {
        $set: { email } });
70     const user = await db.collection('users')
71       .findOne({ username }, { projection: { _id: 0, username
        : 1, email: 1 } });
72     res.json({ success: true, user });
73   } catch (err) { res.status(500).json({ error: 'Erreur
        serveur' }); }
74 });
```

C. Checklist de sécurité — API Node.js + MongoDB

Table C.1 – Vecteurs d’attaque, opérateurs et défenses

Vecteur	Opérateur	Défense principale	Défense secondaire
Contournement login	<code>\$ne: ""</code>	Validation de type : rejeter tout objet sur champ password	Comparaison JS côté serveur
Exfiltration collection	<code>\$regex: ""</code>	Validation de format + <code>safeFilter</code>	Projection minimale + <code>limit()</code>
Sélection étendue	<code>\$gt: ""</code>	Validation de type + allow-list	<code>safeFilter</code> avec champs nommés
Mass Assignment	Champs injectés dans <code>\$set</code>	Jamais <code>\$set:req.body</code>	Allow-list des champs modifiables
Fuite de champs	Absence de projection	Projection minimale sur tous les <code>find()</code>	Chiffrement des champs sensibles